Sharing page tables with mshare()

This article brought to you by LWN subscribers

Subscribers to LWN.net made this article — and everything that surrounds it — possible. If you appreciate our content, please buy a subscription and make the next set of articles possible.

By **Jonathan Corbet**
May 17, 2022

LSFMM

The Linux kernel allows processes to share pages in memory, but the page tables used to control that sharing are not, themselves, shared; as a result, processes sharing memory maintain duplicate copies of the page-table data. Normally this duplication imposes little overhead, but there are situations where it can hurt. At the 2022 Linux Storage, Filesystem, Memory-management and BPF Summit (LSFMM), Khaled Aziz (remotely) and Matthew Wilcox led a session to discuss a proposed mechanism to allow those page tables to be shared between cooperating processes.

Some `mshare()` background

There was not much discussion of the motivation for this work or the proposed API in this session, which was focused on implementation. That information can be found, though, in this patch set posted in April. Eight bytes of page-table entry per page is not much overhead — until you have thousands of processes sharing the page, at which point the space taken by page tables is more than the shared page itself. There are applications out there that run that many processes, so there is a desire to reduce the overhead of non-shared page tables.

The proposal is a pair of new system calls, the first of which is `mshare()`:

```
int mshare(char *name, void *addr, size_t length, int oflags, mode_t mode);
```

A process wanting to share a range of memory (along with the page tables) will first create a region, probably with `mmap()`; this region must be aligned to a 512GB boundary. The call to `mshare()` provides the address and size of this region, along with a `name` to identify it. This call, if successful, will create a file with the given `name` under `/sys/fs/mshare` that, when read, will provide the given `addr` and `length` values.

Any other process that wishes to share this region of memory will start by opening that file and reading the associated address and size; it can then call `mshare()` with that information to set up the mapping. The permissions on the file in `/sys/fs/mshare` control the access to this region. The mapping shares the memory, but also the page tables that control it. As a result, any changes to those page tables, with `mmap()` or `mprotect()` for example, will affect all processes that are sharing the region.

When a process is finished with the shared area, it can call `mshare_unlink()`, passing the given `name`; when all processes detach from the region, it will be destroyed.

Wilcox began the session by noting that a process's address space is described by `struct mm_struct`, of which each process has one. When `mshare()` is used to create a shared area, a new `mm_struct` is created to describe that part of the address space. This structure has no tasks assigned to it, but it is pointed to from the virtual memory areas (VMAs) in each process that have the area mapped. Since one process's actions on the shared area affect all of them, this mechanism is suitable for cooperating processes that trust each other.

Scary

Aziz had a set of questions for the group. What, he asked, is the right granularity for page-table sharing? The current patch set shares page tables at the PMD level, but there might be value in sharing higher-level page directories. He asked whether the proposed API makes sense, and whether it should be possible for a process to map only a portion of the shared region (which is not supported now). Should `mremap()` be supported in a shared region? He also had questions about how `userfaultfd()` should interact with this feature.

Michal Hocko started by saying that this feature "sounds scary". He had a number of questions of his own. Who, in the end, is in charge of the shared `mm_struct` structure? How is memory accounting handled? What about mapping with the `MAP_FIXED` flag (used by a process that wants to tell the kernel where in its address space a mapping should be placed)? Wilcox answered that, for the most part, this mapping is handled in the same way as a mapping shared by threads within a single process. Aziz said that a worry of his own is that the shared area might be useful for processes trying to hide malware. Before getting into that sort of issue, though, he asked whether the `mshare()` concept seems useful in general.

Mike Rapoport asked why the `SCM_RIGHTS` mechanism, which allows passing file descriptors over a Unix-domain socket, wasn't used to control access to the shared region. Wilcox answered that the first design for this feature did exactly that, but users were requesting the ability to open a file to access the area instead. John Hubbard said that the API looked elegant to him, and requested that the developers stick with it.

Dan Williams asked how page pinning and accounting were being handled; Aziz replied that the work was mostly focused on the basic functionality so far. Making `get_user_pages()` and such work was on the list of things to do, though. David Hildenbrand echoed Hocko's sentiment that the feature seemed scary; he suggested making an allowlist describing the actions that were permitted on a shared area. System calls like `mlock()` would not be on that list, he suggested, until the implications were well understood. Page pinning, too, should not be there at the outset, he said.

Wilcox said that the users driving this work want to use it with DAX (direct access to files stored in persistent memory). These users can have over 10,000 processes sharing the area, which causes the page-table overhead to exceed the amount of memory being shared. In a sense, he said, `mshare()` can be seen as giving DAX the same functionality as hugetlbfs, but nobody likes hugetlbfs, so the desire is to make something that is not so awful. Hocko suggested that the new API is "a different awful".

Continuing, Wilcox said that, with `mshare()`, the kernel now has the concept of a standalone `mm_struct` with a file descriptor attached to it. What else, he asked, could be done with that functionality? Perhaps there would be value in a more general system call that would create an `mm_struct` and allow processes to attach things to it. That would be an interesting concept, he said, but Hildenbrand suggested it would be something more like Frankenstein's monster. Wilcox responded that Frankenstein would have loved this idea; he was "a misunderstood genius, just like us".

API alternatives

Hubbard suggested that perhaps a different model would make more sense; it could be called a "lightweight process" (or just a "Frankenstein"). These new processes would have a set of rules describing their behavior. But Hocko said that he couldn't understand the consequences of such a feature; they would be "beyond imagination", he said. He asked why processes can't just share page tables on a per-mapping basis, using a feature that looks like hugetlbfs but in a more shareable way. Wilcox answered that "the customer" wants the described semantics where, for example, `mprotect()` applies across all processes, just as if they were threads sharing that part of the address space. That raises an obvious question, he said: why not just use threads? The answer was that "`mmap_lock` sucks". It is also not possible to change the existing behavior of `MAP_SHARED`, since that would break programs, so there would need to be, at a minimum, a new `mmap()` flag if not a new system call. Aziz said that the separate system call makes the page-table sharing explicit rather than it just being a side effect. That makes the decision to opt into this behavior explicit as well.

Liam Howlett asked how many `mshare()` regions are supported in any given process; Wilcox answered that there is no particular limit. A process can create as many files as it wants, but he does not expect the API to be used that way. A more typical pattern would be for processes to share a single large chunk of memory, then perhaps map pieces of it. Howlett responded that, in that case, it might be better to only allow a single region per process. That might simplify the impact on other parts of the memory-management subsystem.

Jason Gunthorpe said that, rather than using a separate `mm_struct`, a process could (via some mechanism) just instantiate a VMA mapped at a high level in the page-table hierarchy. The associated memory would be owned by that VMA (or the inode of a file backing it), and the reference counting could be done there. Hocko noted that this is how hugetlbfs works now. Wilcox answered that an explicit opt-in from the processes involved is still needed, since developers need to understand the changed semantics of system calls like `mprotect()`. Gunthorpe suggested a new `mmap()` flag. Aziz said that an approach like this was possible, but that the use of a separate `mm_struct` has the advantage of simplifying the use of existing mechanisms for working with page tables.

Wilcox started to wind down the session by saying that, if the memory-management developers found this idea too scary, something else could be done. Aziz said that he was about to send the next version of the patch set (which hasn't happened as of this writing) and he would see what the feedback is at that point.

As things were coming to a close, Jan Kara jumped in to say that the `mmap_lock` for the shared region will have the same contention problems as it does now. Wilcox said that he knew somebody would bring that up; to an extent, that problem does exist. But `mshare()` allows processes to have more than one memory region and separate private memory from shared memory. The effect, he said, is like splitting `mmap_lock` in half. But even separating out 20% of the contention, he said, would be an improvement. Kara asked whether it might be better, instead, to give threads a way to separate their *private* address space. Wilcox said that he had thought the same way a year ago, but the result in the end is about the same. Kara said that the concept might be easier for developers to grasp.

At that point the session came to an end for real. The next step will be further discussion on the mailing list once the updated patch set comes out.

Index entries for this article

Kernel	Huge pages
Kernel	Memory management/Page-table sharing
Kernel	System calls/mshare()
Conference	Storage, Filesystem, Memory-Management and BPF Summit/2022

Log in to post comments

[–] Sharing page tables with mshare()
Posted May 17, 2022 19:41 UTC (Tue) by **dezgeg** (subscriber, #92243) [Link] (1 responses)

> This call, if successful, will create a file with the given name under /sys/fs/mshare that, when read, will provide the given addr and length values.

How would this interact with namespaces (containers)? For example, would container A be able to see mshare'd regions created in container B (thus information leak)?

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 18, 2022 2:53 UTC (Wed) by **willy** (subscriber, #9762) [Link]

Haven't really been thinking about containers. Off the top of my head, msharefs would need to be mounted in each container as separate instances.

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 18, 2022 6:12 UTC (Wed) by **roc** (subscriber, #30627) [Link]

It would be pretty interesting if a shared region could be copied with copy-on-write semantics.

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 18, 2022 13:55 UTC (Wed) by **ncm** (guest, #165) [Link] (2 responses)

This doesn't seem to make sense.

Why not have the downstream processes just mmap the mshare file, and let the kernel work out the rest?

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 18, 2022 18:05 UTC (Wed) by **willy** (subscriber, #9762) [Link] (1 responses)

Because the app wants different semantics from those normally provided by mmap(). Specifically, setting mprotect() on a sub-region of the area needs to apply to all processes mapping the area, not just this one.

It really does want "Treat this region of address space as if the sharing processes are threads but the rest of the address space is mine".

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 18, 2022 22:33 UTC (Wed) by **zev** (subscriber, #88455) [Link]

Though this plus a new opt-in mmap() flag is essentially what it sounds like Jason Gunthorpe asked about? Or have I misunderstood something?

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 18, 2022 14:53 UTC (Wed) by **jstarks** (guest, #117831) [Link] (1 responses)

Would be great to see an fd-based interface for mapping even if msharefs is still present to simplify coordination. This would be useful for mutually cooperating sandboxed processes, where most of the processes don't have access to msharefs (via whatever mechanism).

A process that did want to make use of msharefs would first open the fd before calling mshare, similar to pidfd functionality.

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 20, 2022 16:44 UTC (Fri) by **kaziz** (subscriber, #117201) [Link]

Something similar was suggested during discussion and I think it is worth pursuing. It also eliminates the need for a system call. I am reworking the patches to support open() and mmap() mechanism using fd. This has significant impact on underlying implementation but it is doable.

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 19, 2022 3:20 UTC (Thu) by **re:fi.64** (subscriber, #132628) [Link] (4 responses)

> this region must be aligned to a 512GB boundary

GB?? If that's not a typo, what's the reason for such a large boundary?

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 19, 2022 4:51 UTC (Thu) by **roc** (subscriber, #30627) [Link] (3 responses)

Presumably because on x86 that's how much virtual address space is covered by an entry in the top-level page table. So you can share the second-level page table associated with that entry in the top-level page tables of multiple processes.

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 20, 2022 16:45 UTC (Fri) by **kaziz** (subscriber, #117201) [Link] (2 responses)

That is exactly right.

Reply to this comment

[–] Sharing page tables with mshare()
Posted Jun 13, 2022 15:40 UTC (Mon) by **nix** (subscriber, #2304) [Link] (1 responses)

So... this has API constraints defined by a single specific architecture? That doesn't sound terribly future-proof. Shouldn't we instead have a scheme where you can ask the kernel what alignment is wanted for these regions, or just ask for a region of some given size and get it back, already appropriately aligned?

Reply to this comment

[–] Sharing page tables with mshare()
Posted Dec 24, 2022 8:37 UTC (Sat) by **dankamongmen** (subscriber, #35141) [Link]

having spent much of my past twenty years mmap()ing, and mremap()ing, and hoping to OS- and architecturally-portably (i.e. without procfs nor sysfs) get available page sizes, and relearning the necessary semantics to reliably use huge pages any given month, mmap(2) is really starting to feel busted.

* if i want some number of virtually-contiguous bytes, i use malloc() (which uses mmap() as a backend for sufficiently large requests, but always the same way: anonymous private).
* if i need them aligned, i use posix_memalign(3).
* if my use case maps to one of madvise(2)'s Borges-like[0] flags, i can give it the ol' college try.
** will it be a no-op this kernel version? who am i to question the will of Allah?
* mremap(2) is there, guaranteeing me job security dealing with subtle linux v freebsd difference
* vmsplice and zerocopy and userfaultfd sometimes come over on the weekends
* MAP_HUGETLB? MAP_HUGE_2MB? MAP_HUGE_1GB? can i provide both to fall back if one isn't available? is hugetlbfs involved? what if i map hugetlbfs without these flags? are hugepages better than superpages? are my smallpages being combined into hugepages by the kernel? is the kernel fracturing my hugepages into smallpages? without kernel command line options can i use them? yeah? without runtime configuration requiring CAP_SYS_ADMIN? so i can't configure them without privs, but i can map them without privs? oh no i need CAP_IPC_LOCK? but i'm not doing IPC? i guess it's a "memory resource" so act like it's an mlock(2)? but i don't need that for regular pages? must every mmap() i write forevermore first call trying to get hugepages, then call again when that fails? how do i know whether the failure was due to hugepages? are a single one of these resource limits tied into the number of largepage TLB entries, probably the most relevant parameter for effective hugetlb use? (these are rhetorical questions; you needn't point me to the answers as of 6.1.1.)

i can't wait for CXL to find its place among this strange brew. there's kernel's mm/ and hardware coherence and the POSIX+ vma/file APIs, and the first two seem more or less on the same page, but the last is at best an imprecise and incomplete means of influencing them.

the mmap(2) flag potpourri is acceptable for sharing and coarse-grained access control on a computer from the early 80s, but as a userspace developer who'd like to use memory effectively on something more complex than an ATARI i want to know:

* what are my memory hierarchies?
* what are my memory-processor-IO topologies?
* what are my cacheline sizes, cache capacities, cache associativities, and page sizes?
* most importantly: given a proposed working set size, where can i keep it in memory?

i want to be able to say "i have a dense 24MB. gimme good memory for that." or "i have 32MB of sparse garbage." or "gimme a sack of buffers that don't alias one another, without my personal study of cache details for this Garbotron 7000". or "i always want these 64KB hot in my TLB but i don't care whether it's a dainty or beefy page beyond that." or "map this file so i can change it in ram and leave things to the page cache, keep it simple, i am eight."

iouring is getting really close with buffer pools and some kernelside dataflow. some kind of buffer coloring seems like it could go a long way here. i did something similar with my libtorque allocator[1], but that project effectively died over a decade ago.

bandwidths and latencies are also interesting, but i'm less likely to design around their precise values (and doing so would lead to a system very sensitive to disruption from other processors). non-temportal stores, IO device streaming into or out of on-die cache, IO device scatter/gather restrictions -- these are all necessary to achieve peak performance. systemwide monitoring of cycles lost to TLB misses could probably be used to configure all the hugetlb stuff better than admins+devs ever could manually. but let's get the simple stuff first.

sorry for the rant, but all this stuff has been the bane of my existence as a userspace hacker for a minute now. it works well enough in rarified HPC environments, where you know and control the machine, but putting out code hoping to use just basic hugetlbs on arbitrary machines is an exercise in annoyance.

[0] https://en.wikipedia.org/wiki/Celestial_Emporium_of_Benev...
[1] https://github.com/dankamongmen/libtorque

Reply to this comment

[–] Sharing page tables with mshare()
Posted Dec 21, 2023 13:39 UTC (Thu) by **derong** (guest, #168675) [Link]

this tech can not only reduce the memory consumed by the page table, but also can reduce the cache miss when page table walking, since fewer mem used, easier to hit in the cache, right?

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 22, 2024 22:29 UTC (Wed) by **TheJH** (subscriber, #101155) [Link] (2 responses)

This could also be fun as a sandboxing technology - you could use mshare() to set up two processes such that some of the address space of process A is mirrored over into a sandboxed process B, and so process A can map memory on process B's behalf, and process B doesn't need the ability to memory-map files by itself anymore.

Almost like the KVM interface where host userspace can map memory into the guest with ioctls, but without all the hardware virtualization complications.

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 22, 2024 22:37 UTC (Wed) by **TheJH** (subscriber, #101155) [Link] (1 responses)

I guess in a way, both KVM and IOMMUv2 are sort of doing a more flexible version of this already, with their mmu-notifier-based mirroring of one process' page tables into other processes? But with the differences that they don't involve creating extra mm_structs, and that the mirroring is a one-way thing.

Reply to this comment

[–] Sharing page tables with mshare()
Posted May 22, 2024 22:39 UTC (Wed) by **TheJH** (subscriber, #101155) [Link]

s/into other processes/into other contexts/

Reply to this comment

Copyright © 2022, Eklektix, Inc.
This article may be redistributed under the terms of the Creative Commons CC BY-SA 4.0 license
Comments and public postings are copyrighted by their creators.
Linux is a registered trademark of Linus Torvalds